

1. `socket(domain, type, protocol)`

Creates the initial file descriptor.

- **domain:** The protocol family. We use **AF_INET** to specify IPv4 (like 192.168.1.1).
- **type:** The communication style. We use **SOCK_STREAM** to specify TCP (reliable, ordered delivery).
- **protocol:** Set this to **0**. This tells the operating system to automatically choose the default protocol for the type you selected.

Eg- `sd=socket(AF_INET, SOCK_STREAM, 0);`

INTERMEDIATE STEPS:

`struct sockaddr_in sad;`

`sad.sin_family = AF_INET;`

`sad.sin_port = htons(9995);`

`sad.sin_addr.s_addr = inet_addr("127.0.0.1");`

2. `bind(sockfd, addr, addrlen)` (Server only)

Links your socket to a specific Port and IP address.

- **sockfd:** The integer ID of the socket you just created (e.g., `server_fd`).
- **addr:** A pointer to a `sockaddr` struct.
- **addrlen:** The size of your struct in bytes, written as `sizeof(address)`.

Eg- `bind(sd, (struct sockaddr *)&sad, sizeof(sad));`

3. listen(sockfd, backlog) (Server only)

Tells the OS to start accepting connections.

- **sockfd**: The server's socket ID.
- **backlog**: An integer representing the queue limit. If you set this to 3, and 4 clients try to connect at the exact same millisecond, the 4th client gets instantly rejected with a "Connection Refused" error.

Eg- `listen(sd,1);`

4. accept(sockfd, addr, addrlen) (Server only)

Halts the program until a client connects, then returns a *new* socket ID.

- **sockfd**: The listening server socket ID.
- **addr**: A pointer to an empty struct where the OS will write the *Client's* IP address so you know who just connected to you.
- **addrlen**: A pointer to an integer where the OS writes the size of the client's struct.

Eg- `struct sockaddr_in cad;`

`cadl = sizeof(cad);`

`cd = accept(sd, (struct sockaddr *)&cad, &cadl);`

5. connect(sockfd, addr, addrlen) (Client only)

Dials out to the server.

- **sockfd**: The client's socket ID.
- **addr**: A pointer to the struct containing the *Server's* IP and Port.
- **addrlen**: The size of that struct, using sizeof().

Eg- `connect(sd, (struct sockaddr *)&sad, sizeof(sad));`

6. send(sockfd, buf, len, flags)

Pushes bytes into the network pipe.

- **sockfd**: The socket ID you are writing to.
- **buf**: A pointer to the data you want to send (usually a string or an array of bytes).
- **len**: The exact number of bytes to send. You usually calculate this with `strlen(message)`.
- **flags**: Special routing options. Almost always set to 0 for standard use.

Eg- `send(sd, str, sizeof(str), 0);`

7. recv(sockfd, buf, len, flags)

Pulls bytes out of the network pipe.

- **sockfd**: The socket ID you are reading from.
- **buf**: A pointer to an empty memory buffer (like a char array) where the OS will deposit the incoming data.
- **len**: The maximum size of your buffer. If your buffer is 1024 bytes, pass 1024 so `recv()` doesn't accidentally overwrite memory outside your array (buffer overflow).
- **flags**: Usually 0.

Eg- `recv(cd, str, sizeof(str), 0);`

** At the end always close the socket , `close(sd)` or `close(cd)`